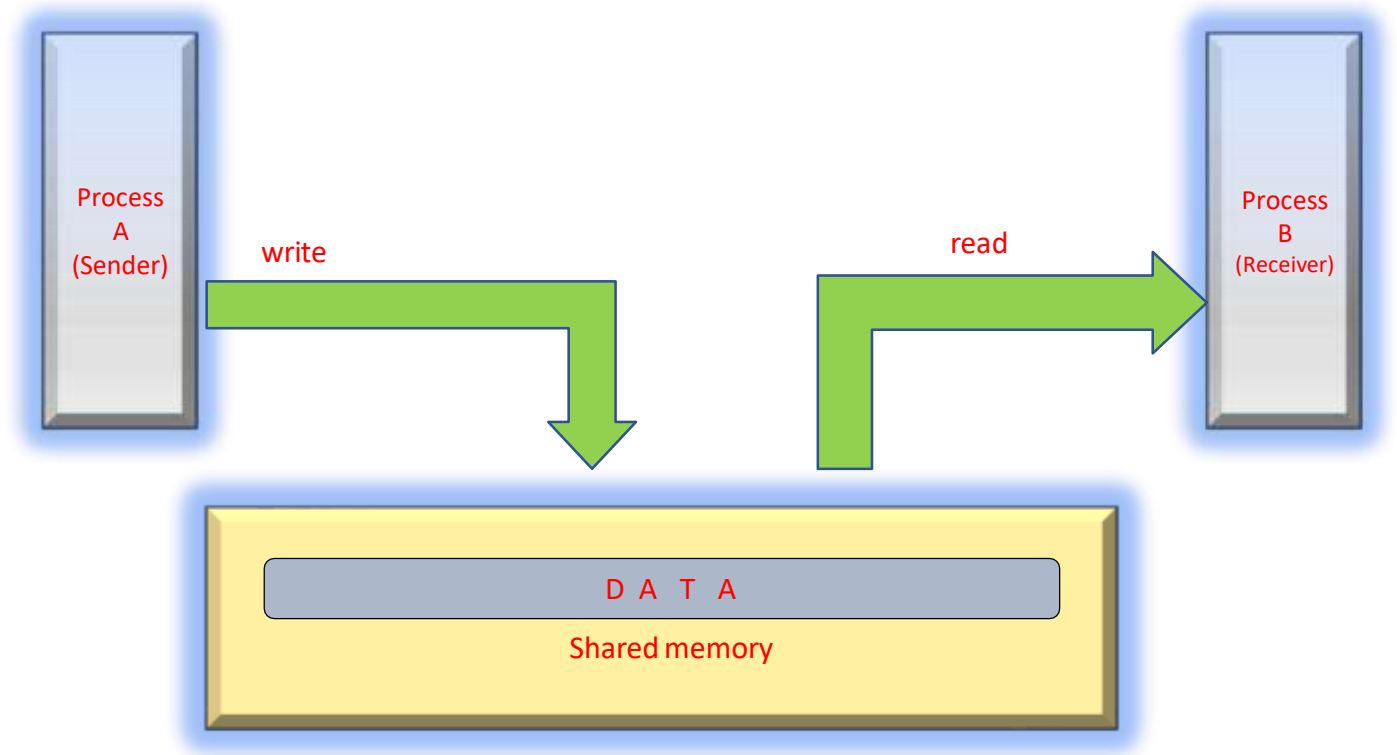


EDAL- IPC Shared Memory

<https://community.ruggedboard.com>

Shared Memory

1. IPC using Shared Memory
2. Shared Memory Concepts
3. Shared memory management APIs
4. Using a shared memory as an IPC



Pre-requisite knowledge

- To understand the concept of Shared Memory, we first need to understand some basics :
 - Virtual Address Space
 - Virtual Memory
 - Program Control Block

What exactly is Virtual Memory and Virtual Address space ?

- Virtual Memory is the total amount of memory your system has for a process.
- It is different from physical memory and is computer architecture dependent

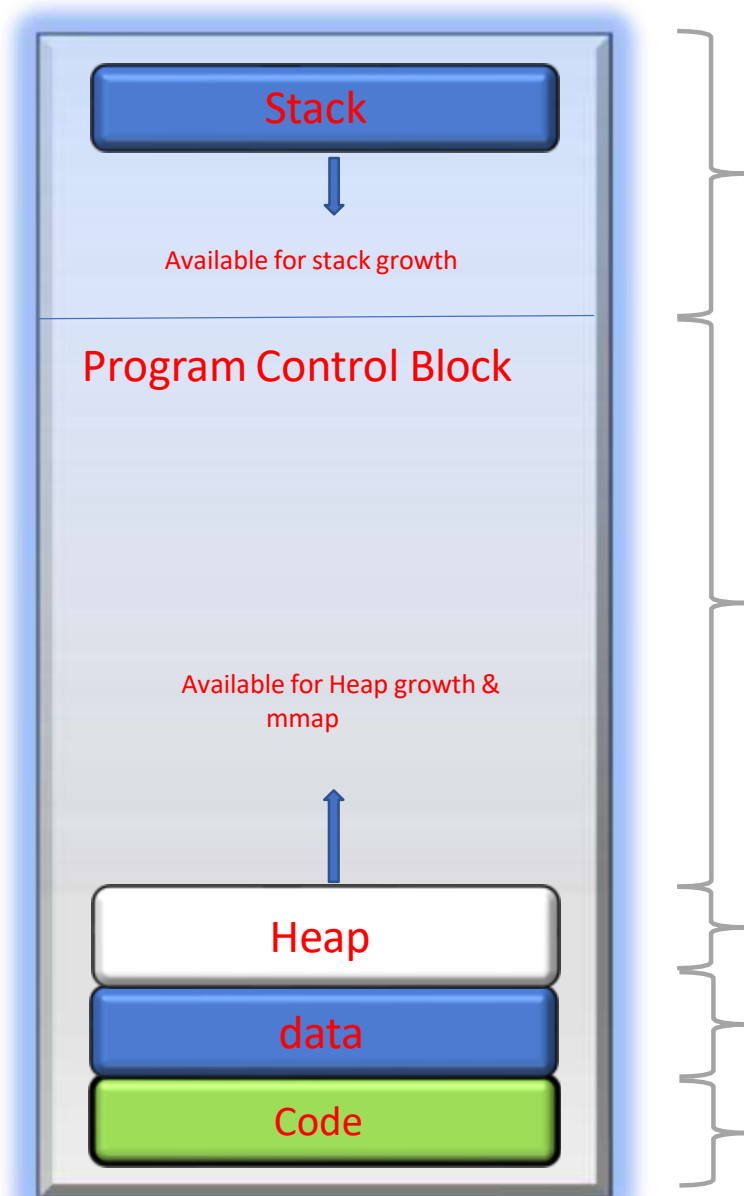
For example :

- For a 32 bit system, Total virtual memory is 2^{32} bytes (Fixed), and it is per process (Not for all processes combined !)
- Whereas You can extend physical memory to 4GB, 8GB or 16GB (Variable) (Upper limit)
- 2^{32} bytes !! Every byte has an address. Therefore, there are 2^{32} Virtual addresses in a 32 bit system.
- Computer Programs works with Virtual Memory only, means, your program access only virtual addresses
- Your Program never knows anything about physical memory , all it knows that it has 2^{32} bytes of virtual memory to store its data
- Each process in execution is allotted virtual memory for its usage, which can grow to maximum of 2^{32} bytes
- The Memory assigned to a process is called process's virtual address space
- Every process running on a 32 bit system, can use maximum of 2^{32} bytes of Virtual memory

Program Control Block

- It is a diagrammatic logical representation of the of the process and its memory layout in Linux/Unix OS
- PCB helps us to understand how process virtual memory works during the course of execution of a process
- Let us see how PCB looks like for a linux process . . .

IPC Shared Memory



Memory assigned for process execution. It cannot grow beyond a certain limit

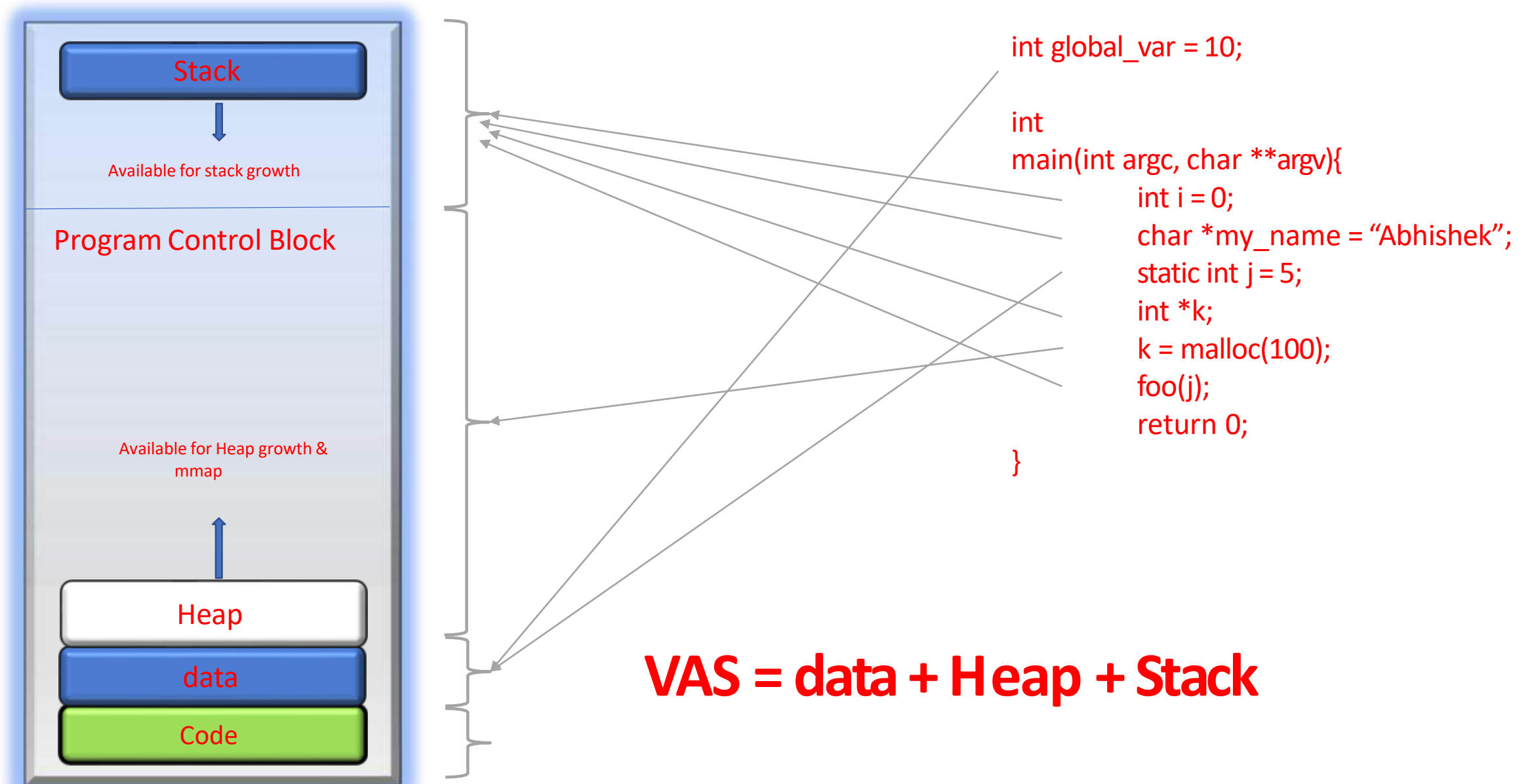
Total System Available memory. The Heap region expands or shrinks When the process malloc Or free the memory

Dynamic memory assigned to a process

Memory which stores the process global and static variables, fixed

Memory which stores the process code in text format, fixed

IPC Shared Memory

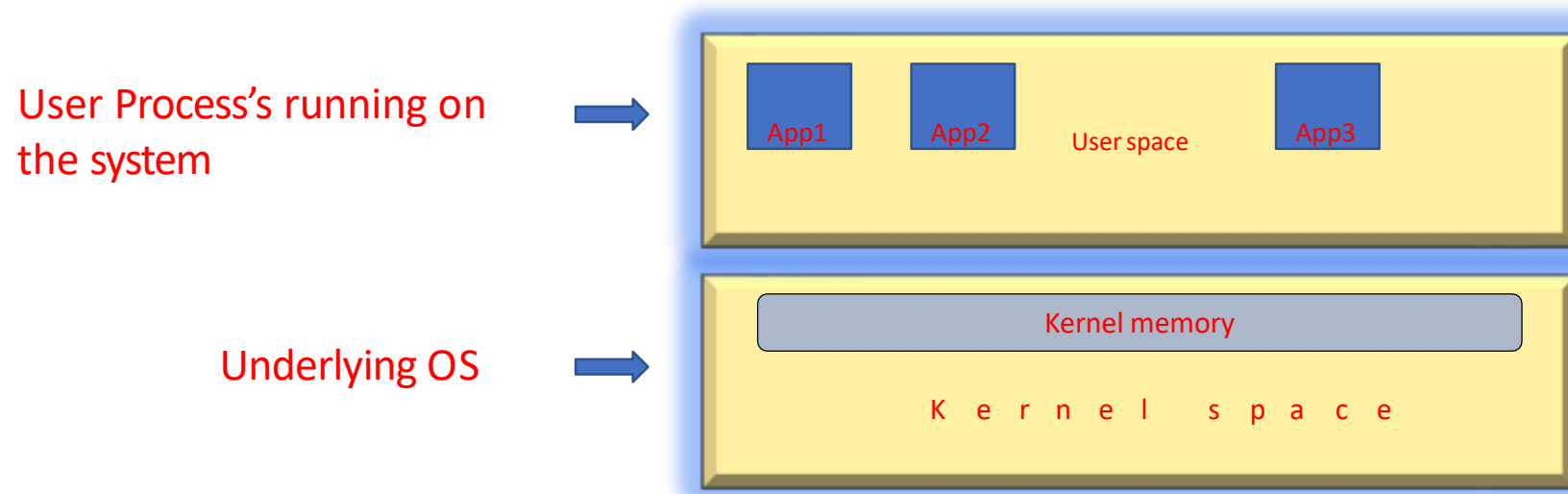


IPC – Techniques -> Shared Memory -> Basics

- Shared Memory is a chunk of Kernel Memory. No process is the owner of this memory.
- In linux, When a process executes, it is assigned a chunk of virtual memory of the system.
This chunk of Memory is called process's virtual address space
- A process can access only the memory addresses which lies within its virtual address space
- If a process tries to access the address which lies outside its virtual address space, Linux OS kills the process
(Segmentation fault)
So, take it as thumb rule – A process is not allowed to read/write into any memory which is outside its virtual memory region
- A process Virtual memory region grows and shrinks as the process request more memory from OS or free the memory
- A given address which lies in virtual memory region of process P1 has no meaning for any other process in the system
- Let us picturize the above stated points ..

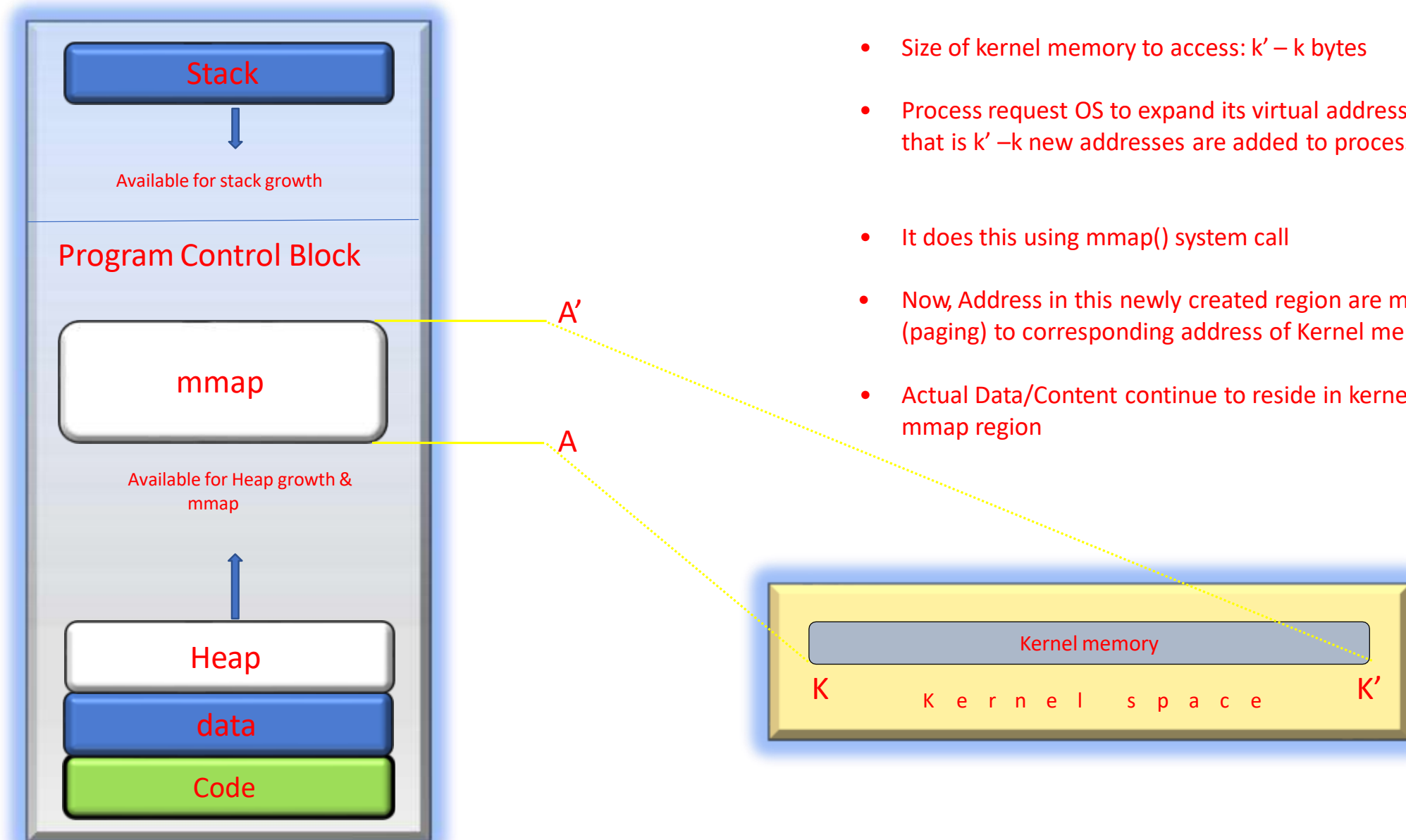
IPC Shared Memory

- Kernel has its own memory called kernel memory. Kernel uses its own memory for its own work – Schedulers, Timers, User process's memory Management, Interrupts and what not !
- No user space process is the owner of this kernel memory
- A user process can request kernel to create a kernel memory region
- This kernel memory region is outside the virtual address space of any process, hence no process can access this memory directly
- Through Shared memory approach, Application(s) can access kernel memory



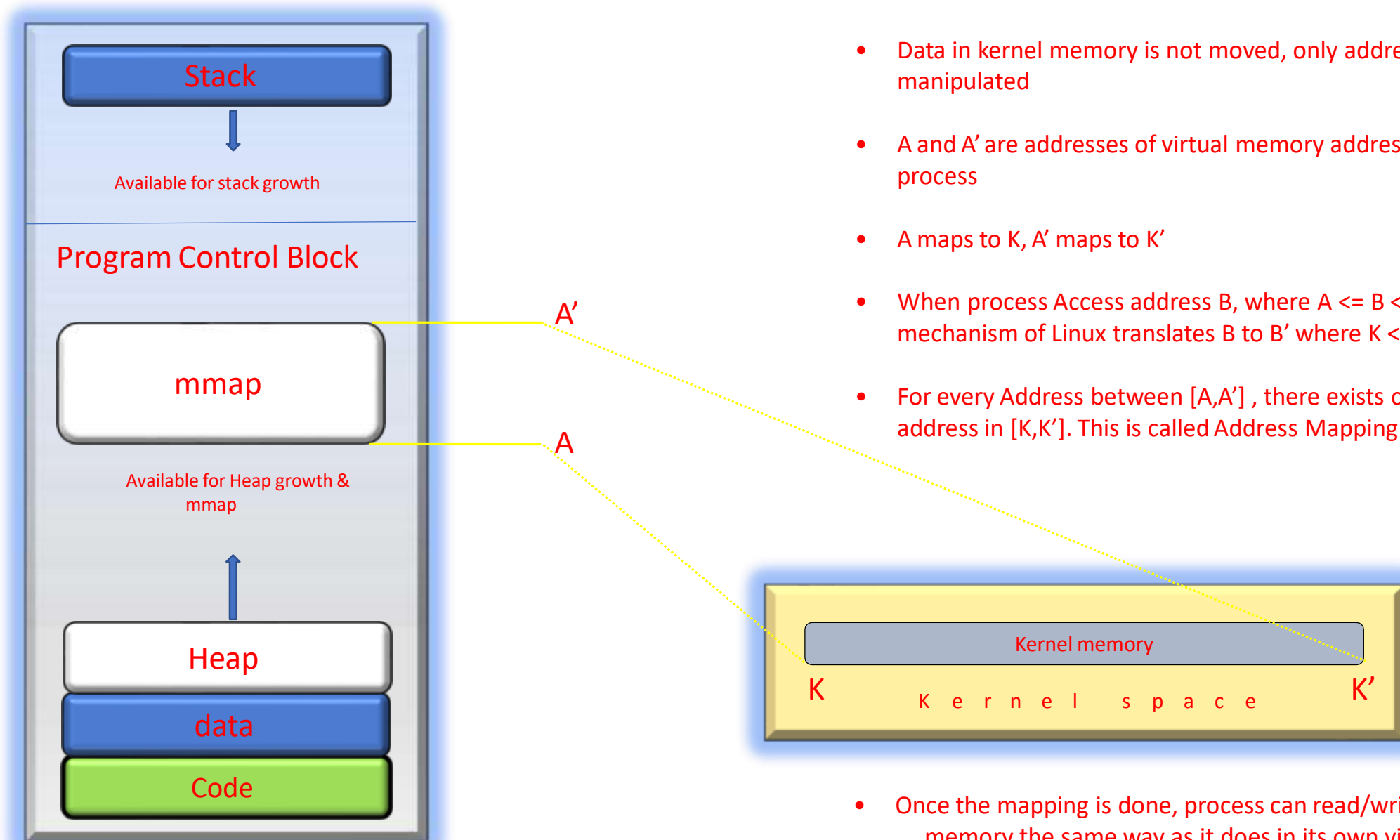
IPC Shared Memory

- Kernel memory starting address : k , end address k'
- Size of kernel memory to access: $k' - k$ bytes
- Process request OS to expand its virtual address space by $k' - k$ bytes , that is $k' - k$ new addresses are added to process VAS from global VAS
- It does this using `mmap()` system call
- Now, Address in this newly created region are mapped by OS (paging) to corresponding address of Kernel memory
- Actual Data/Content continue to reside in kernel memory, not in new `mmap` region



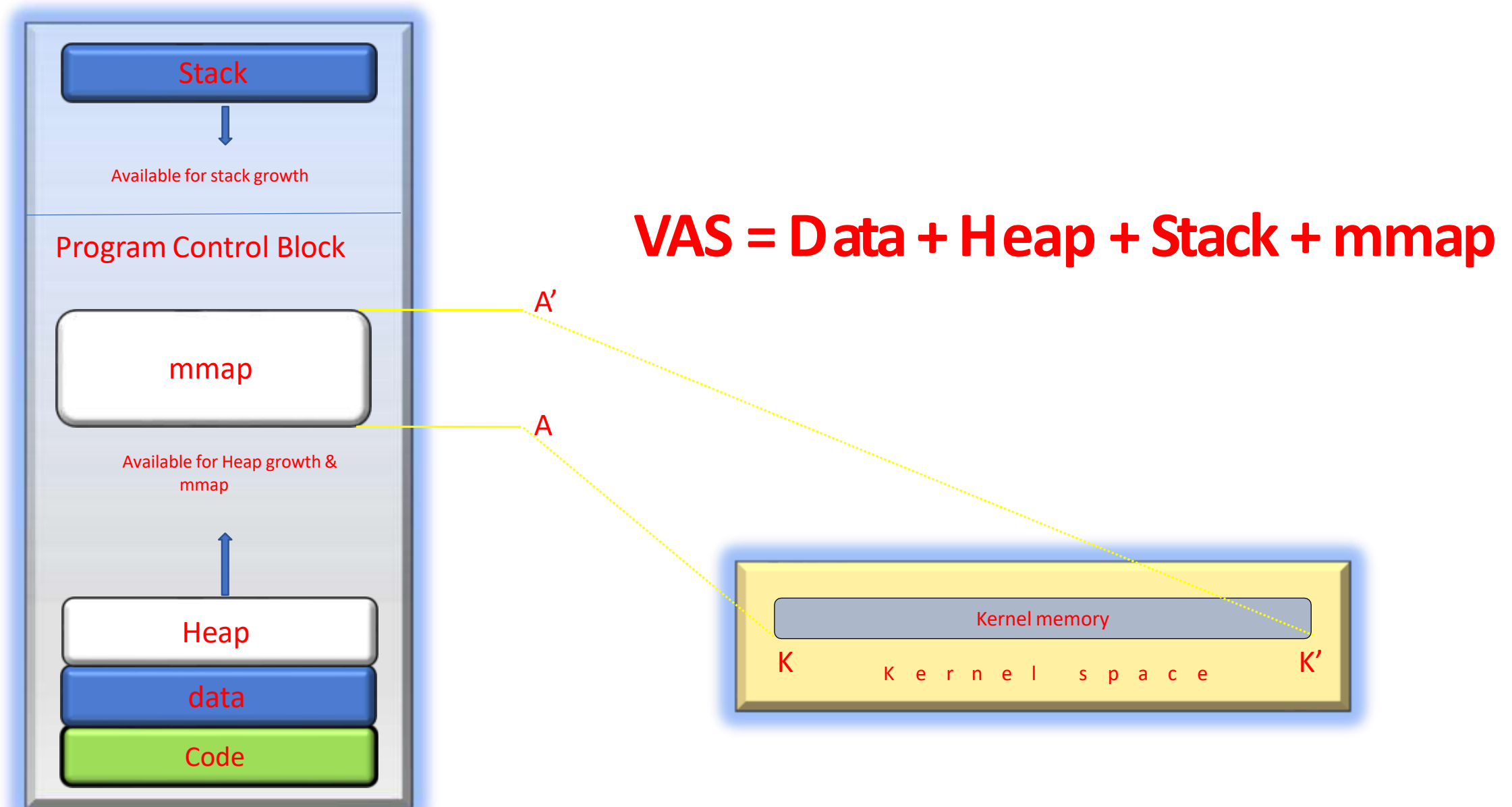
IPC Shared Memory

- $A' - A = K' - K$
- Data in kernel memory is not moved, only address mappings is manipulated
- A and A' are addresses of virtual memory address space of process
- A maps to K, A' maps to K'
- When process Access address B, where $A \leq B < A'$, then paging mechanism of Linux translates B to B' where $K \leq B' < K'$
- For every Address between [A,A'], there exists corresponding address in [K,K']. This is called Address Mapping



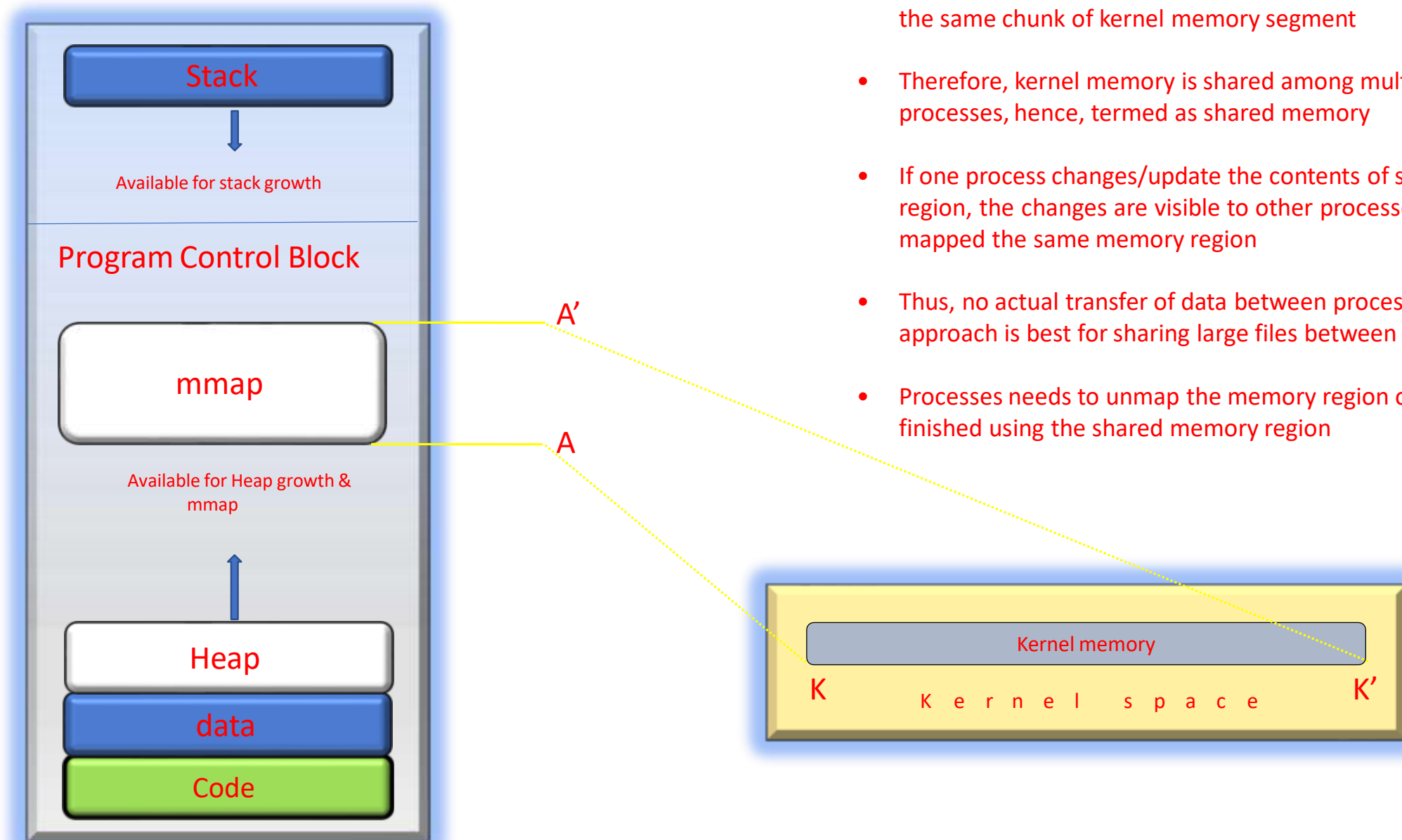
- Once the mapping is done, process can read/write into kernel memory the same way as it does in its own virtual memory

IPC Shared Memory

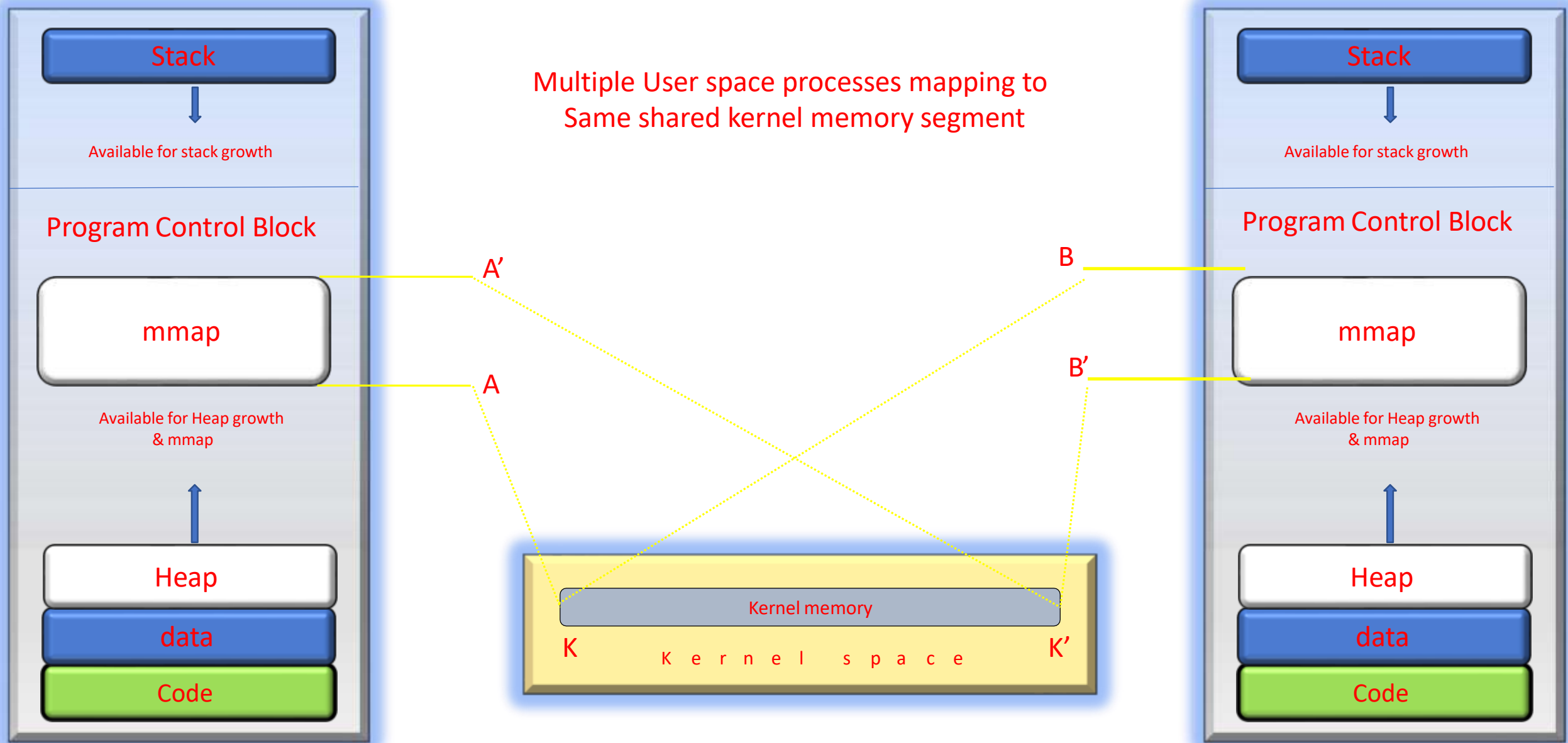


IPC Shared Memory

- Two or more processes can do this mapping at the same time with the same chunk of kernel memory segment
- Therefore, kernel memory is shared among multiple user space processes, hence, termed as shared memory
- If one process changes/update the contents of shared memory region, the changes are visible to other processes which have mapped the same memory region
- Thus, no actual transfer of data between processes, hence, this approach is best for sharing large files between processes
- Processes need to unmap the memory region once it is finished using the shared memory region



IPC Shared Memory



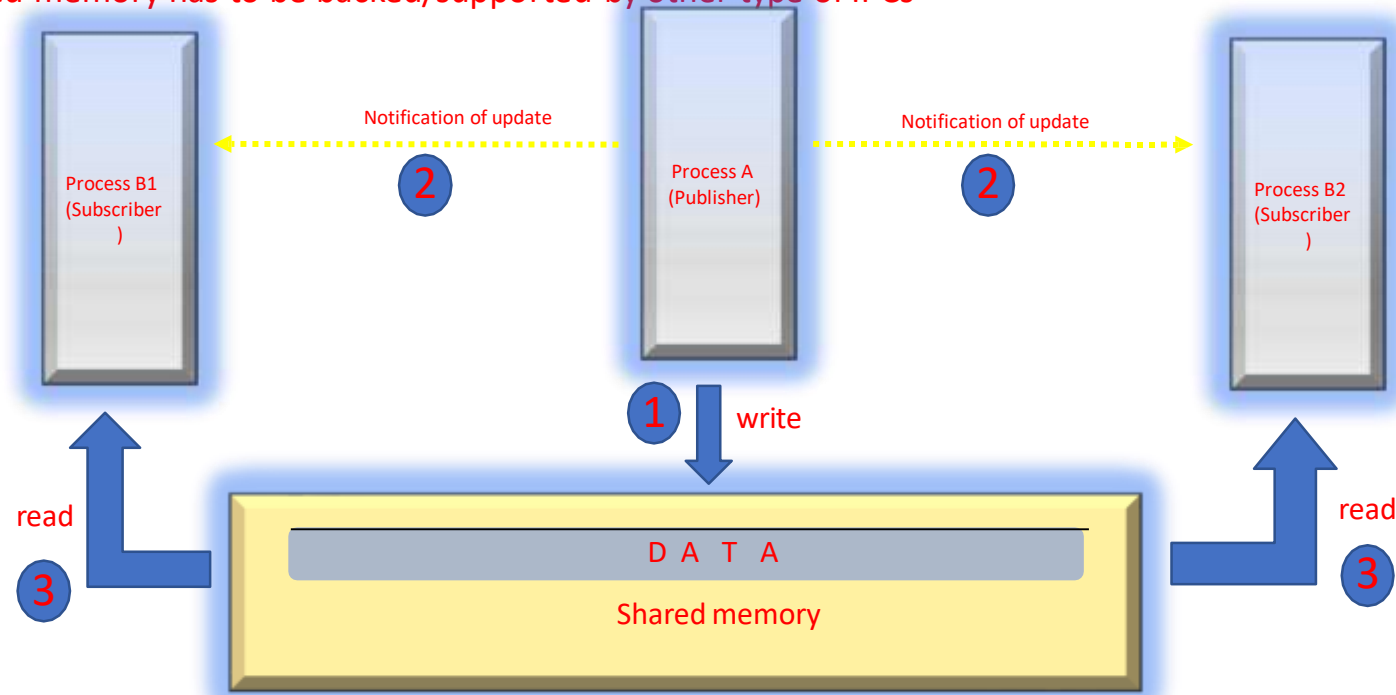
Design Constraints for using Shared Memory as IPC

- **Recommendation when it is good to use Shared Memory IPC :**
 - Shared Memory approach of IPC is used in a scenario where :
 - Exactly one processes is responsible to update the shared memory (publisher process)
 - Rest of the processes only read the shared memory (Subscriber processes)
 - The frequency of updating the shared memory by publisher process should not be very high
 - For example, publisher process update the shared memory when user configure something on the software
 - If multiple processes attempts to update the shared memory at the same time, then it leads to write-write conflict :
 - We need to handle this situation using Mutual Exclusion based Synchronization
 - Synchronization comes at the cost of performance
 - Because we put the threads to sleep (in addition to their natural CPU preemption) in order to prevent concurrent access to critical section

IPC Shared Memory

- **When publisher process update the shared memory :**

- The subscribers would not know about this update
- Therefore, After updating the shared memory, publisher needs to send a notification to all subscribers which states that “shared memory has been updated”
- After receiving this notification, Subscribers can read the updated shared memory and update their internal data structures, if any
- The notification is just a small message, and can be sent out using other IPC mechanisms, such as Unix domain sockets Or Msg Queues
- Thus IPC using shared memory has to be backed/supported by other type of IPCs



Shared Memory APIs

1. Creating Or opening a Shared memory region

```
int  
shm_open (const char *name, int oflag, mode_t mode);
```



Name of shared memory
Eg `"/Data_segment"`

Eg :

```
int shmfd =  
    shm_open("/datasegment", O_CREAT | O_RDWR | O_TRUNC,  
            0660);
```

- Create a new shared memory object inside a kernel
- returns an integer which represents a file descriptor to this shared memory object
- returns -1 if call fails
- *oflag* – Following flags , OR-ing of one or more flags is supported
 - `O_CREAT` -- create a shared memory if it don't exist already
 - `O_RDONLY` – open a shared memory object in read only mode
 - `O_WRONLY` - open a shared memory object in write only mode
 - `O_RDWR` - open a shared memory object in read-write mode
 - `O_APPEND` – New data shall be appended to the existing content of shared memory
 - `O_TRUNC` – If specified, it truncates the shared memory object, if already exists, to zero bytes again
 - If `O_EXCL` flag is used together with the `O_CREAT` flag, then *shm_open* fails if shared shared memory object already exists with the same name
- *mode* – if the shared memory is newly created by the process, then mode is the file permissions applied on this memory object
- A newly created shared memory object is of size zero bytes
- It can be made of the desired size by using the *ftruncate* system call

2. Setting the size of shared memory object

```
int  
ftruncate (int shm_fd, size_t new_size);
```

- Resize the shared memory object to new size
- Returns -1 on failure
- Should be called after *shm_open* if required.

3. Mapping the shared memory into process's Virtual Address Space

```
#include  
<sys/mman.h> void *  
mmap (void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

offset is the location in the shared memory object at which the mapping starts; we will use the value zero for offset and map the shared memory object starting right from the beginning

fd is, of course, the file descriptor for the shared memory received from an earlier `shm_open` call

`MAP_SHARED`.
which means that the updates to the mapped shared memory are visible to all other processes immediately

`PROT_READ | PROT_WRITE`

length of shared memory object that should be mapped

Starting address of process's VAS where Shared memory object shall begin mapping
Recommended to Pass as `NULL`. Let OS decide.

On success, `mmap` returns the pointer to the location in Its virtual address space where the shared memory region has been mapped.

In case of error, `MAP_FAILED`, which is, `(void *) - 1`, is returned and `errno` is set to the cause of the error.

4. Un-Mapping the shared Memory

int

munmap (void *addr, size_t length);

- *addr* – Address in process virtual address space where Shared memory has been mapped (starting end of mapping) (Return value of mmap())
- *length* – length of mapping which we want to unmap
- unmapps the shared memory object at location pointed by *addr* and having size, *length*
- Returns -1 on failure, 0 on success
- It do not destroys the shared memory, it only destroy the mapping between shared memory in kernel space and virtual memory of the process

5. shm_unlink

```
int  
shm_unlink (const char *name);
```

- De-associate shared memory with its name
- It is equivalent to like no shared memory with the same name exists anymore. The name shall be available to associate with new instance of shared memory
- return -1 on error, 0 on success

6. closing the shared memory file descriptor

```
int  
close (int shm_fd)
```

- decrease the reference count of the shared memory object maintained in kernel space
- If reference count decreases to zero, shared memory and its content is destroyed (on some systems continue to exists until system is rebooted)
- return -1 on error, 0 on success

Points to remember

- Shared memory objects open and/or mapped by a process are both closed and unmapped when that process terminates prematurely, but they are not automatically unlinked.
- They will persist at least until manually unlinked or the system is rebooted.

Problems

Thank You



Developer
Wiki





Attribution 4.0 International (CC BY 4.0)

This is a human-readable summary of (and not a substitute for) the [license](#). [Disclaimer](#).

You are free to:

Share — copy and redistribute the material in any medium or format

Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

